



UNIVERSIDAD VERACRUZANA



FACULTAD DE INSTRUMENTACIÓN ELECTRÓNICA

PROGRAMA EDUCATIVO:
INGENIERÍA EN INSTRUMENTACIÓN ELECTRÓNICA

EXPERIENCIA EDUCATIVA:
SISTEMAS DIGITALES

FACILITADOR:
JOSÉ ALFONSO DOMINGUEZ CHAVEZ

NOMBRES DE LOS ESTUDIANTES:
AGUILAR ORTIZ VICTOR
BELTRAN AMEZCUA GABRIEL
RONZÓN PÉREZ AXEL DE JESÚS
SANCHEZ PEREZ JONATHAN

MATRICULAS DE LOS ESTUDIANTES:
S24013180
S24013131
S24013170
S24013135

FECHA:
1 DE JUNIO DE 2026

DESCRIPCIÓN DEL PROYECTO:

El presente proyecto consiste en el diseño e implementación de un carro seguidor de línea controlado mediante una FPGA Tang Nano 9K. El sistema tiene como finalidad detectar y seguir una trayectoria marcada sobre una superficie mediante sensores infrarrojos, realizando correcciones automáticas en la dirección para mantener el recorrido establecido.

El núcleo de control del sistema está basado en una **FPGA GW1NR-LV9QN88PC6/15 (Tang Nano 9K)**, programada en lenguaje Verilog. La FPGA recibe las señales provenientes de tres sensores infrarrojos encargados de detectar la posición de la línea respecto al vehículo. Con base en esta información, el sistema determina la acción correspondiente para cada motor, permitiendo avanzar en línea recta, corregir la trayectoria hacia la izquierda o derecha, o realizar una maniobra de reversa cuando sea necesario.

Como mecanismo de seguridad e inicio, se incorporó un sensor LDR (Light Dependent Resistor), el cual permanece monitoreando la intensidad luminosa del entorno. El vehículo inicia su funcionamiento únicamente cuando el sensor detecta un destello de luz (flash), evitando activaciones accidentales durante la preparación o manipulación del sistema.

El movimiento del carro es realizado mediante dos motores de corriente directa controlados por un puente H L298N. Este módulo permite invertir el sentido de giro de los motores y regular su funcionamiento a través de señales digitales generadas por la FPGA. Además, se implementó una señal PWM (Pulse Width Modulation) con un contador de 8 bits que limita de manera constante la potencia de los motores al 60% de su capacidad total (ciclo de trabajo de 153/255), garantizando un desplazamiento suave, estable y eficiente.

La estructura física del proyecto está compuesta por un chasis móvil equipado con dos ruedas motrices, una rueda loca para soporte y estabilidad, un compartimento para cuatro baterías AA que suministran energía al sistema, un interruptor de encendido, cableado mediante jumpers y los sensores necesarios para la navegación autónoma.

Los componentes utilizados para la construcción del proyecto fueron:

- FPGA Tang Nano 9K (GW1NR-LV9QN88PC6/15).
- Puente H L298N.
- Dos motores de corriente directa con sus respectivas ruedas.
- Portapilas para cuatro baterías AA.

- Interruptor de encendido.
- Tres sensores infrarrojos para detección de línea.
- Un sensor LDR para activación mediante flash.
- Una rueda loca de soporte.
- Chasis para montaje de componentes.
- Cables jumper hembra-hembra para las interconexiones.

El costo aproximado de los materiales utilizados para la construcción del prototipo fue de \$594 MXN.

Mediante este proyecto se aplicaron conocimientos de electrónica digital, diseño de sistemas lógicos programables, lenguaje de descripción de hardware Verilog, control de motores mediante puente H y procesamiento de señales provenientes de sensores, integrando hardware y software en una aplicación de robótica móvil autónoma.

TABLA DE LA VERDAD:

Los sensores infrarrojos trabajan con lógica activa en bajo, es decir, cuando un sensor detecta la línea negra entrega un valor lógico 0, mientras que cuando no detecta la línea entrega un valor lógico 1.

La FPGA analiza continuamente el estado de los tres sensores y genera las señales correspondientes para el control de los motores mediante el puente H L298N.

S1	S2	S3	Combinación Binaria	Acción del vehículo	Motor Izquierdo	Motor Derecho
0	0	0	3'b000	Reversa	Reversa	Reversa
0	0	1	3'b001	Giro a la derecha	Adelante	Apagado
0	1	0	3'b010	Avanza recto	Adelante	Adelante
0	1	1	3'b011	Giro a la derecha	Adelante	Apagado
1	0	0	3'b100	Giro a la izquierda	Apagado	Adelante
1	0	1	3'b101	Detenido	Apagado	Apagado
1	1	0	3'b110	Giro a la izquierda	Apagado	Adelante
1	1	1	3'b111	Detenido	Apagado	Apagado

Los casos 111 y 101 no se asignaron a una acción específica ya que no son relevantes para las acciones del carro.

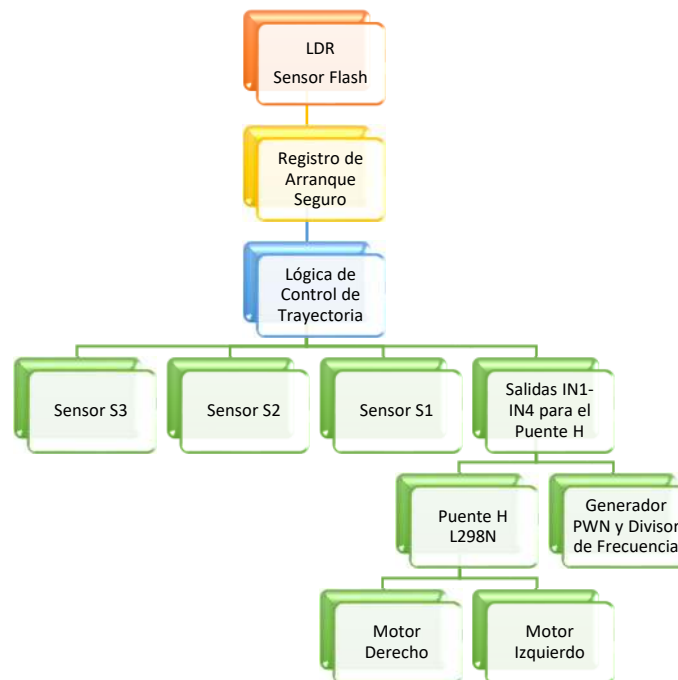
Tabla de salidas del puente H:

Estado	IN1	IN2	IN3	IN4
Avance recto	pulso_pwm	0	pulso_pwm	0
Giro a la derecha	pulso_pwm	0	0	0
Giro a la izquierda	0	0	pulso_pwm	0
Reversa	0	pulso_pwm	0	pulso_pwm
Detenido	0	0	0	0

DIAGRAMA LÓGICO:

El sistema fue implementado mediante una arquitectura modular dentro de la FPGA Tang Nano 9K. La FPGA recibe las señales provenientes de los sensores infrarrojos y del sensor LDR, procesa la información mediante lógica digital y genera las señales de control necesarias para el puente H L298N encargado de accionar los motores del vehículo.

El funcionamiento general del sistema se muestra en el siguiente diagrama de bloques:

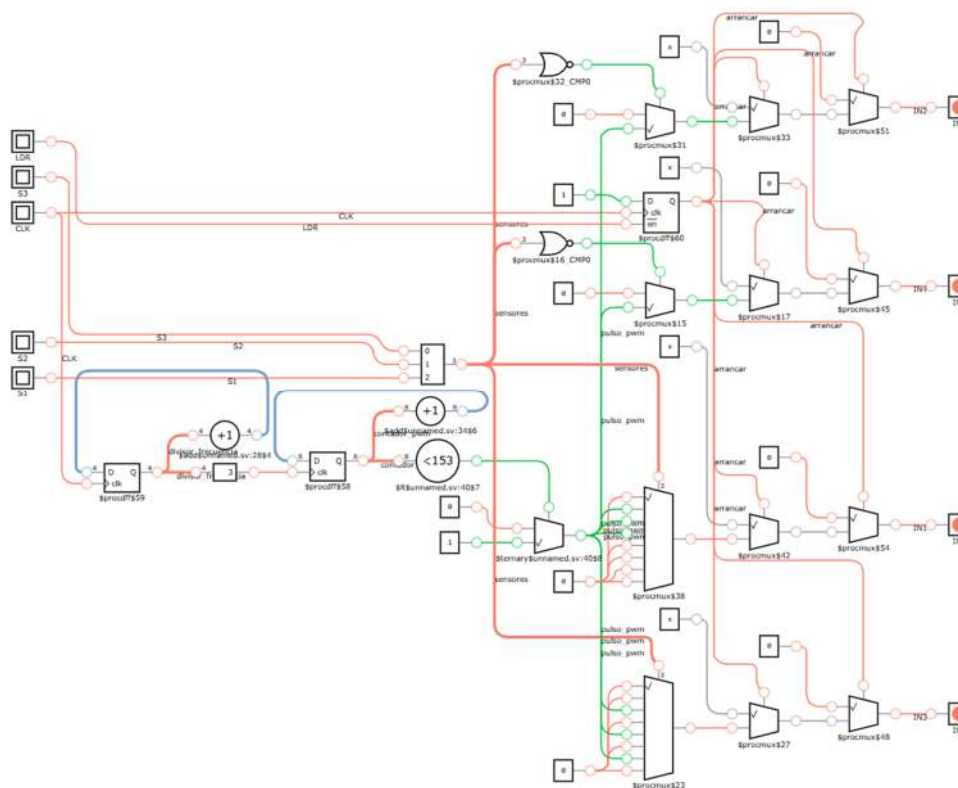


Descripción de los Bloques

1. **Sensor LDR:** Detecta un destello luminoso (flash) que habilita el inicio del sistema.

2. **Registro de Arranque Seguro:** Almacena el evento detectado por el LDR y permite que el carro comience su funcionamiento.
3. **Sensores Infrarrojos S1, S2 y S3:** Detectan la posición relativa de la línea respecto al vehículo.
4. **Lógica de Control de Trayectoria:** Evalúa continuamente las combinaciones de los sensores infrarrojos y determina la acción correspondiente para los motores: avanzar en línea recta, realizar correcciones hacia la izquierda o derecha, ejecutar maniobras de recuperación mediante reversa o detener el vehículo en condiciones no válidas de detección.
5. **Divisor de Frecuencia y Generador PWM:** Reduce la frecuencia del reloj interno de la FPGA y genera una señal PWM para controlar la velocidad de los motores.
6. **Puente H L298N:** Recibe las señales de control provenientes de la FPGA y suministra la potencia necesaria para el movimiento de los motores.
7. **Motores de Corriente Directa:** Proporcionan el desplazamiento del vehículo y permiten realizar los cambios de dirección indicados por la FPGA.

Diagrama lógico creado en el programa de DigitalJS Online:



Descripción del diagrama generado por la simulación:

La simulación y síntesis visual del código Verilog en DigitalJS Online permite validar cómo el sintetizador interpreta las descripciones de hardware y las convierte en celdas lógicas, multiplexores y registros reales:

- **Bloque de Tiempo y PWM (Zona inferior izquierda):** Se observa el registro síncrono \$procdff\$59 que corresponde al divisor_frecuencia de 4 bits (con su respectivo sumador de hardware +1). El bit más significativo (bit 3) se conecta directamente al reloj del segundo registro secuencial \$procdff\$58, el cual actúa como el contador_pwm de 8 bits. La salida de este contador se conecta a un comparador de magnitud (< 153), el cual conmuta una señal binaria (línea verde: pulso_pwm) que determina el ciclo de trabajo del 60%.
- **Registro de Arranque por Flash (Zona superior central):** El sensor LDR controla la habilitación del flip-flop con entrada de reloj y habilitación (Clock Enable) etiquetado como \$procdff\$60. Este registro almacena de forma persistente el valor lógico 1 (bandera arrancar) en cuanto se detecta el primer flanco por flasheo.
- **Lógica de Conmutación de Trayectoria (Zona central y derecha):** Las señales provenientes de los sensores infrarrojos (S1, S2, S3) se agrupan en un bus de 3 bits (sensores) que actúa como la línea de selección de los multiplexores principales (\$procmux\$38, \$procmux\$23, entre otros). Estos multiplexores rutean dinámicamente la señal del pulso_pwm o valores de tierra (0) hacia las salidas físicas de los motores.
- **Etapas de Seguridad Final:** Antes de salir hacia los pines físicos (IN1 a IN4), las señales atraviesan una última cascada de multiplexores controlados por la señal global arrancar. Si arrancar es 0, las salidas se fuerzan inmediatamente a cero lógico (0), garantizando el aislamiento eléctrico y el paro total de los motores por seguridad.

CÓDIGO DEL DISEÑO:

El diseño del carro seguidor de línea fue desarrollado en lenguaje Verilog y está compuesto por cuatro bloques principales: sistema de arranque mediante LDR, divisor de frecuencia, generador PWM y lógica de control de los motores.

```
module carro_seguidor(  
    input CLK,          //Señal de reloj interna de la FPGA  
    input S1,           //Sensor Izquierdo (0 = detecta línea, 1 = NO detecta)  
    input S2,           //Sensor Central   (0 = detecta línea, 1 = NO detecta)
```

```

    input S3,          //Sensor Derecho  (0 = detecta línea, 1 = NO detecta)
    input LDR,         //Sensor de inicio (0 = detecta Flash)

    output reg IN1,    //Motor Izquierdo A
    output reg IN2,    //Motor Izquierdo B
    output reg IN3,    //Motor Derecho A
    output reg IN4     //Motor Derecho B
);
//Bandera de control para habilitar el movimiento general del circuito
reg arrancar = 0;

//Registro del Flash para el arranque seguro
always @(posedge CLK) begin
    if (LDR == 0)      //Si el LDR recibe luz del flash, se activa
        permanentemente
            arrancar <= 1;
end

//=====
//GENERADOR DE PWM CON DIVISOR DE FRECUENCIA
//=====
//Divisor para ralentizar el reloj original y que el Puente H responda
reg [3:0] divisor_frecuencia = 0;
always @(posedge CLK) begin
    divisor_frecuencia <= divisor_frecuencia + 4'd1; //Contador síncrono de 4
bits
end

//El contador de PWM avanza al ritmo del bit más alto del divisor
reg [7:0] contador_pwm = 0;
always @(posedge divisor_frecuencia[3]) begin
    contador_pwm <= contador_pwm + 8'd1;    //Escalado de reloj para bajar la
frecuencia del PWM
end

//Ajuste de la velocidad del carro (153 es el 60% de la potencia del carro. El
máximo es 255)

```

```

wire [7:0] velocidad = 8'd153;
//Generación del ciclo de trabajo (Duty Cycle) mediante comparación binaria
wire pulso_pwm = (contador_pwm < velocidad) ? 1'b1 : 1'b0;

//=====
//LÓGICA DE CONTROL DE LOS MOTORES (CASOS DE SENSORES)
//=====
//Agrupamos los sensores: [S1, S2, S3]
wire [2:0] sensores = {S1, S2, S3};

always @(*) begin
    //Valores por defecto (Motores apagados)
    IN1 = 0; IN2 = 0;
    IN3 = 0; IN4 = 0;

    if(arrancar) begin //El carro solo opera si la bandera de inicio fue
        activada por el LDR
        case (sensores)

            //=====
            //CASO: Izquierdo y Derecho SÍ, Central NO (010)
            //Ambos motores avanzan al mismo tiempo
            //=====
            3'b010: begin
                IN1 = pulso_pwm; IN2 = 0; //Motor Izquierdo Adelante
                IN3 = pulso_pwm; IN4 = 0; //Motor Derecho Adelante
            end

            //=====
            //CASO: Izquierdo NO detecta, central y derecho SÍ (100)
            //Giro a la Izquierda (Arranca Derecho, apaga izquierdo)
            //=====
            3'b100: begin
                IN1 = 0; IN2 = 0; //Motor Izquierdo Apagado
                IN3 = pulso_pwm; IN4 = 0; //Motor Derecho Adelante
            end

```



```

//=====
//CASO: Derecho NO detecta, central e izquierdo SÍ (001)
//Giro a la Derecha (Arranca Izquierdo, apaga derecho)
//=====
3'b001: begin
    IN1 = pulso_pwm; IN2 = 0; //Motor Izquierdo Adelante
    IN3 = 0; IN4 = 0;        //Motor Derecho Apagado
end

//=====
//CASO MODIFICADO: Únicamente el derecho detecta algo (110)
//El motor derecho avanza y el izquierdo NO
//=====
3'b110: begin
    IN1 = 0; IN2 = 0;        //Motor Izquierdo Apagado
    IN3 = pulso_pwm; IN4 = 0; //Motor Derecho Adelante
end

//=====
//CASO MODIFICADO: Únicamente el izquierdo detecta algo (011)
//El motor izquierdo avanza y el derecho NO
//=====
3'b011: begin
    IN1 = pulso_pwm; IN2 = 0; //Motor Izquierdo Adelante
    IN3 = 0; IN4 = 0;        //Motor Derecho Apagado
end

//=====
//CASO: ÚNICAMENTE los 3 sensores detectan (000)
//Ambos motores en REVERSA al mismo tiempo
//=====
3'b000: begin
    IN1 = 0; IN2 = pulso_pwm; //Motor Izquierdo Reversa
    IN3 = 0; IN4 = pulso_pwm; //Motor Derecho Reversa
end

//=====

```

```

        //CASO POR DEFECTO: Apaga motores por seguridad ante
        //Cualquier combinación no programada (evita latches).
        //=====
        default: begin
            IN1 = 0; IN2 = 0;
            IN3 = 0; IN4 = 0;
        end
    endcase
end
endmodule

```

CÓDIGO DEL TESTBENCH:

Con el propósito de verificar el correcto funcionamiento del diseño antes de su implementación física en la FPGA, se desarrolló un banco de pruebas (Testbench) en lenguaje Verilog. El testbench permite generar estímulos de entrada controlados para simular las diferentes condiciones de operación del carro seguidor de línea y observar la respuesta de las señales de salida asociadas al puente H L298N.

La simulación fue realizada mediante la herramienta DigitalJS Online, la cual permitió visualizar el comportamiento temporal de las señales digitales y comprobar que las acciones programadas para cada combinación de sensores coincidieran con el funcionamiento esperado del sistema.

```

`timescale 1ns/1ps //Definición de la unidad de tiempo (1ns) y precisión de la
simulación (1ps)

```

```

module carro_seguidor_tb;

```

```

//=====
//ENTRADAS DEL DUT (Device Under Test) - Declaradas como reg
//=====
reg CLK; //Señal de reloj del sistema
reg S1; //Estímulo para Sensor Izquierdo (0 = detecta línea, 1 = NO detecta)
reg S2; //Estímulo para Sensor Central (0 = detecta línea, 1 = NO detecta)
reg S3; //Estímulo para Sensor Derecho (0 = detecta línea, 1 = NO detecta)
reg LDR; //Estímulo para el sensor óptico de arranque (0 = detecta Flash)

```

```

//=====
//SALIDAS DEL DUT - Declaradas como wire para observar los cambios
//=====
wire IN1; //Estado del Motor Izquierdo Adelante (PWM)
wire IN2; //Estado del Motor Izquierdo Reversa (PWM)
wire IN3; //Estado del Motor Derecho Adelante (PWM)
wire IN4; //Estado del Motor Derecho Reversa (PWM)

//=====
//INSTANCIA DEL MÓDULO A PROBAR (UUT - Unit Under Test)
//=====
carro_seguidor uut (
    .CLK(CLK),
    .S1(S1),
    .S2(S2),
    .S3(S3),
    .LDR(LDR),
    .IN1(IN1),
    .IN2(IN2),
    .IN3(IN3),
    .IN4(IN4)
);

//=====
//GENERADOR DE RELOJ
//Genera un flanco cada 5ns, logrando un Periodo = 10ns (100 MHz)
//=====
always #5 CLK = ~CLK;

//=====
//BLOQUE PRINCIPAL DE ESTÍMULOS DE PRUEBA
//=====
initial begin
    //--- ESTADO INICIAL ---
    CLK = 0; //Inicializa el reloj en bajo

```

```

    LDR = 1; //El carro inicia en modo de espera (LDR en reposo, no ha detectado
flash)
    S1 = 1; //Sensores inicialmente sobre la superficie clara / fuera de línea
(Leen 1)
    S2 = 1;
    S3 = 1;

#50; //Espera de estabilización inicial del sistema de 50 nanosegundos

//--- ACTIVACIÓN MEDIANTE FLASH (LDR) ---
    LDR = 0; //Envía un pulso en bajo (Simula el destello del Flash sobre el
fotorresistor)
    #20;      //Mantiene el pulso de luz por 20ns para asegurar la captura
síncrona
    LDR = 1; //El flash termina. La bandera interna 'arrancar' ya debe fijarse
en 1 de forma permanente.

//--- CASO 1: AVANCE RECTO (3'b010) ---
    //Solo el sensor central (S2) detecta la línea negra (0). S1 y S3 permanecen
en blanco (1).
    S1 = 1; S2 = 0; S3 = 1;
    #100; // Evalúa el comportamiento: Ambos motores deben avanzar hacia
adelante (IN1 y IN3 con PWM)

//--- CASO 2: GIRO CRÍTICO A LA IZQUIERDA (3'b100) ---
    //El carro se desvió a la derecha; S1 pierde el camino, S2 y S3 pisan la
línea negra.
    S1 = 1; S2 = 0; S3 = 0;
    #100; //El motor derecho avanza con PWM (IN3) para corregir el rumbo, el
izquierdo se apaga (IN1=0).

//--- CASO 3: GIRO CRÍTICO A LA DERECHA (3'b001) ---
    //El carro se desvió a la izquierda; S3 pierde el camino, S1 y S2 pisan la
línea negra.
    S1 = 0; S2 = 0; S3 = 1;
    #100; //El motor izquierdo avanza con PWM (IN1) para corregir el rumbo, el
derecho se apaga (IN3=0).

```

```
//--- CASO 4: CORRECCIÓN SUAVE A LA IZQUIERDA (3'b110) ---  
//El carro empieza a salirse por la derecha; únicamente el sensor derecho  
(S3) toca la línea negra.  
S1 = 1; S2 = 1; S3 = 0;  
#100; //El motor derecho avanza con PWM (IN3) e izquierdo apagado para  
reincorporarse al centro.
```

```
//--- CASO 5: CORRECCIÓN SUAVE A LA DERECHA (3'b011) ---  
//El carro empieza a salirse por la izquierda; únicamente el sensor  
izquierdo (S1) toca la línea negra.  
S1 = 0; S2 = 1; S3 = 1;  
#100; //El motor izquierdo avanza con PWM (IN1) e derecho apagado para  
reincorporarse al centro.
```

```
//--- CASO 6: REVERSA POR ENCRUCIJADA O PISTA PERDIDA (3'b000) ---  
//Los 3 sensores detectan negro simultáneamente (intersección completa o  
anomalía).  
S1 = 0; S2 = 0; S3 = 0;  
#100; //Ambos motores invierten su giro a velocidad controlada (IN2 y IN4  
con PWM).
```

```
//--- CASO 7: ESTADO ANÓMALO / CLÁUSULA DEFAULT (3'b101) ---  
//El sensor central lee blanco (1) y los laterales negro (0). Imposible en  
condiciones normales.  
S1 = 0; S2 = 1; S3 = 0;  
#100; //Entra a la zona de seguridad 'default': Ambos motores se  
desenergizan por completo (0).
```

```
//--- CASO 8: PISTA PERDIDA POR COMPLETO / FUERA DE LÍNEA (3'b111) ---  
//Todos los sensores leen la superficie clara (1). El carro perdió la pista.  
S1 = 1; S2 = 1; S3 = 1;  
#100; //Entra a la zona 'default': Los motores permanecen apagados para  
evitar que el carro escape.
```

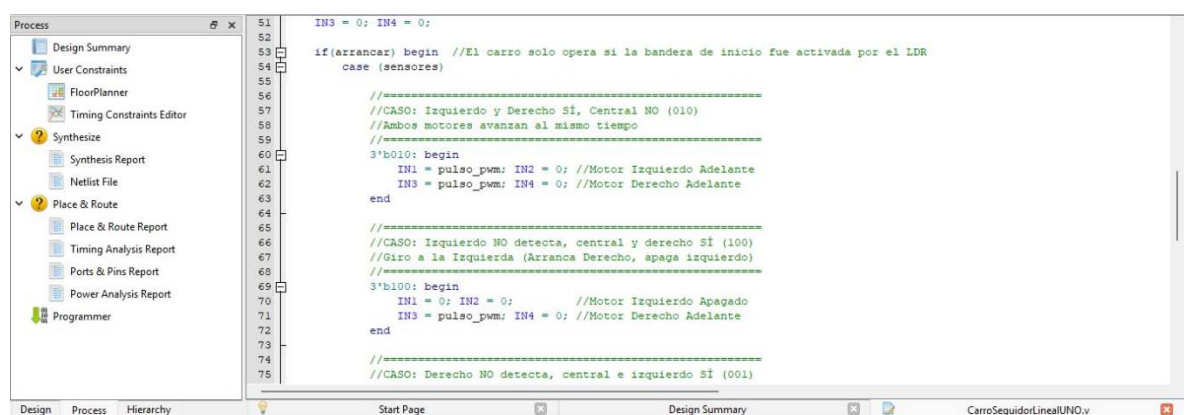
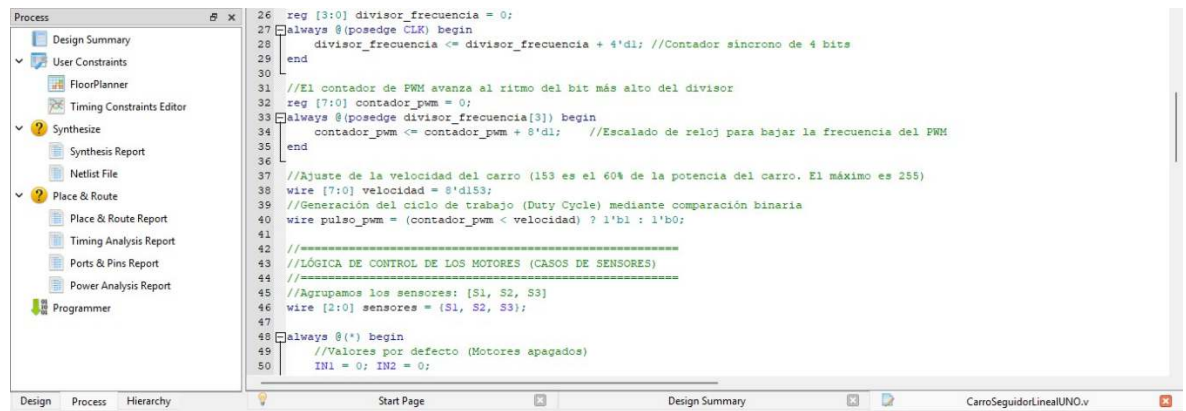
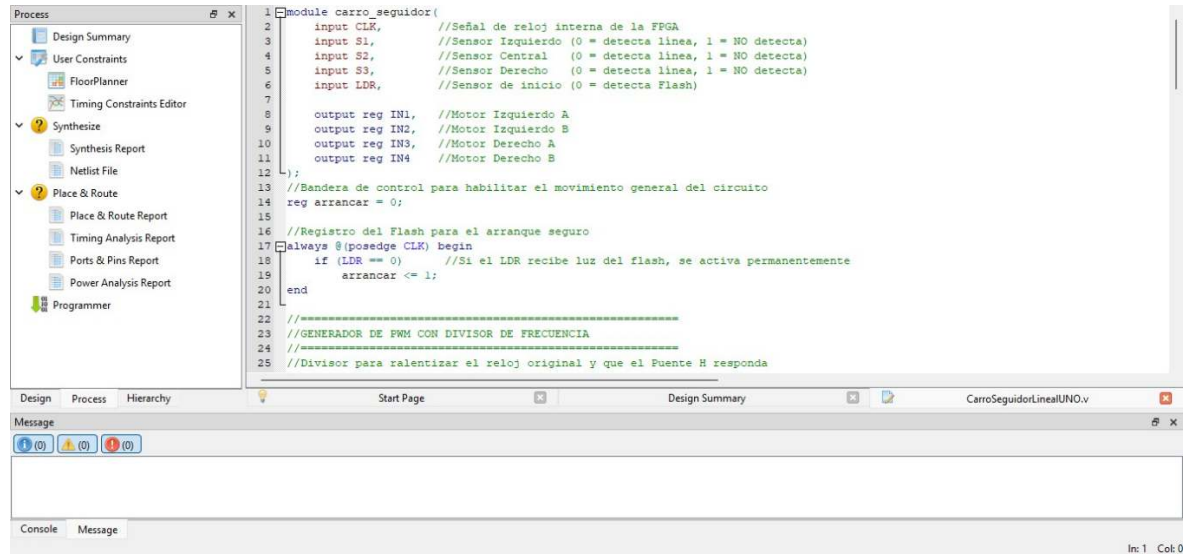
```
//--- FINALIZACIÓN DE LA SIMULACIÓN ---
```

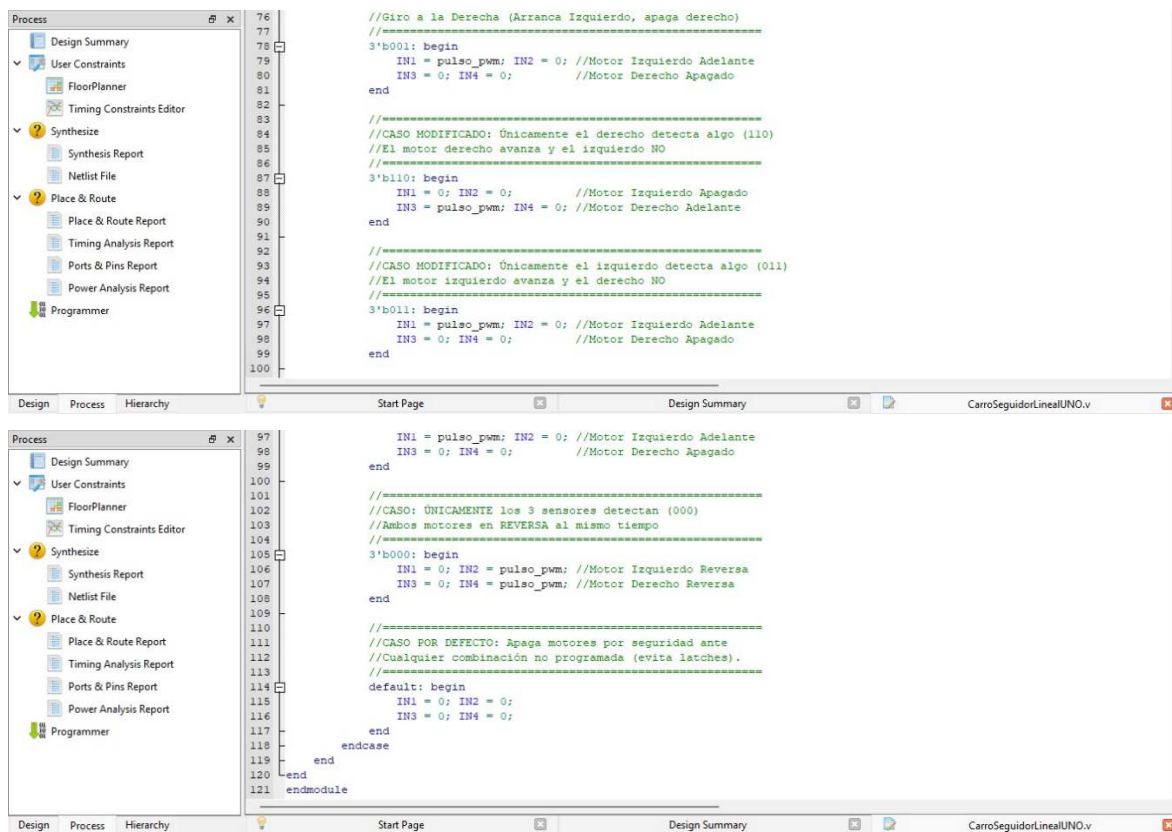
```

$finish; //Termina la sesión de simulación en el entorno EDA (ModelSim,
Questa, DigitalJS, etc.)
end
endmodule

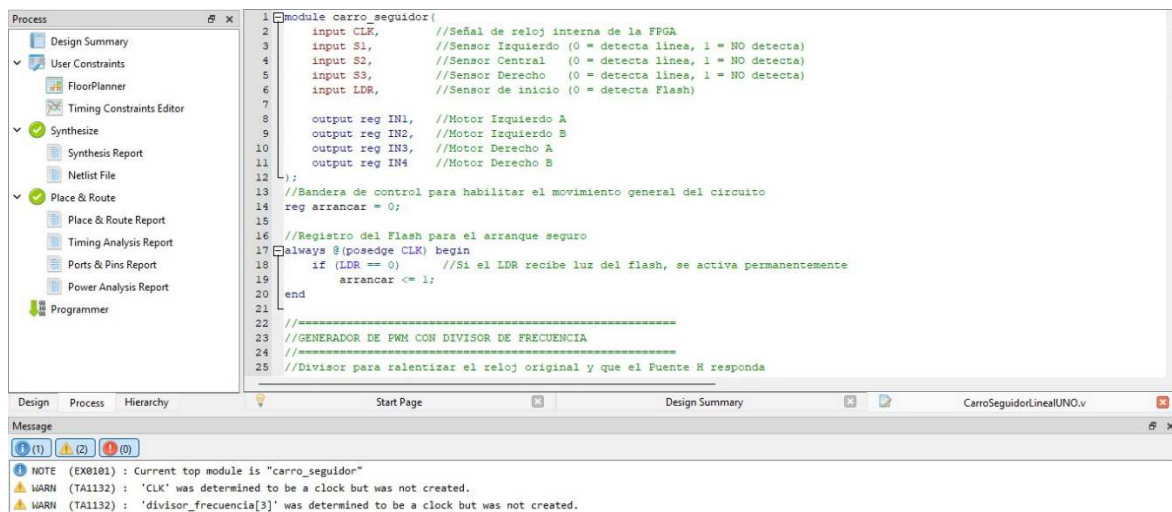
```

CAPTURAS DE LA SIMULACIÓN:





CAPTURA DE SÍNTESIS:



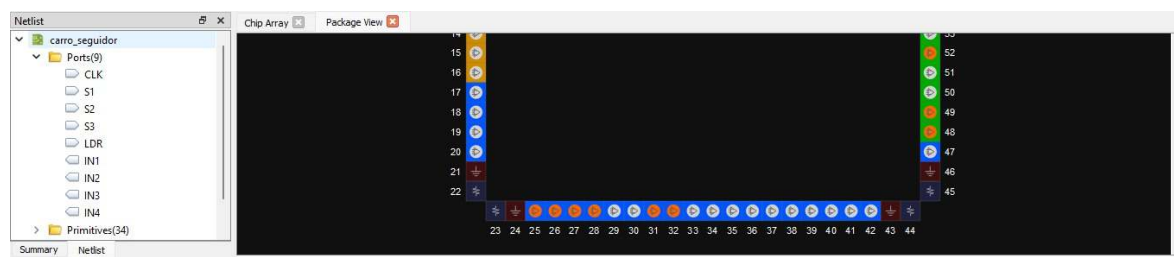
Durante el proceso de síntesis e implementación en Gowin FPGA Designer se generaron las siguientes advertencias de análisis de tiempos (TA):

- WARN (TA1132): 'CLK' was determined to be a clock but was not created.
- WARN (TA1132): 'divisor_frecuencia[3]' was determined to be a clock but was not created.

Estas advertencias no representan errores sintácticos ni lógicos, por lo que no impidieron la generación del *bitstream* ni el funcionamiento correcto del sistema. El software Gowin FloorPlanner detectó que las señales CLK y divisor_frecuencia[3] actúan directamente como fuentes de disparo para bloques secuenciales (`always @(posedge ...)`); sin embargo, reporta que no se han definido de manera formal sus frecuencias de operación mediante un archivo de restricciones de temporización **SDC (Synopsys Design Constraints)**.

La primera advertencia corresponde al oscilador de reloj principal de la tarjeta de desarrollo. La segunda aparece porque el bit más significativo (bit 3) del registro divisor se utilizó como un reloj secundario generado por lógica para ralentizar el contador del PWM. A pesar de que el analizador de tiempos trabaja a ciegas respecto a los retardos de estas líneas, el desfase en nanosegundos es mecánicamente imperceptible, permitiendo que el diseño fuera sintetizado, implementado y programado exitosamente en la FPGA **Tang Nano 9K** con un comportamiento óptimo en la pista.

MAPA DE LOS PINES:



	Port	Direction	Diff Pair	Location	Bank	Exclusive	IO Type	Drive	Pull Mode	PCI Clamp	Hysteresis	Open Drain	Vref	Sing
1	S3	input		48	1	False	LVC MOS33	N/A	NONE	ON	NONE	N/A	N/A	
2	S2	input		32	2	False	LVC MOS33	N/A	NONE	ON	NONE	N/A	N/A	
3	S1	input		31	2	False	LVC MOS33	N/A	NONE	ON	NONE	N/A	N/A	
4	LDR	input		49	1	False	LVC MOS33	N/A	NONE	ON	NONE	N/A	N/A	
5	IN4	output		28	2	False	LVC MOS33	8	NONE	N/A	N/A	OFF	N/A	
6	IN3	output		27	2	False	LVC MOS33	8	NONE	N/A	N/A	OFF	N/A	
7	IN2	output		26	2	False	LVC MOS33	8	NONE	N/A	N/A	OFF	N/A	
8	IN1	output		25	2	False	LVC MOS33	8	NONE	N/A	N/A	OFF	N/A	
9	CLK	input		52	1	False	LVC MOS33	N/A	NONE	ON	NONE	N/A	N/A	

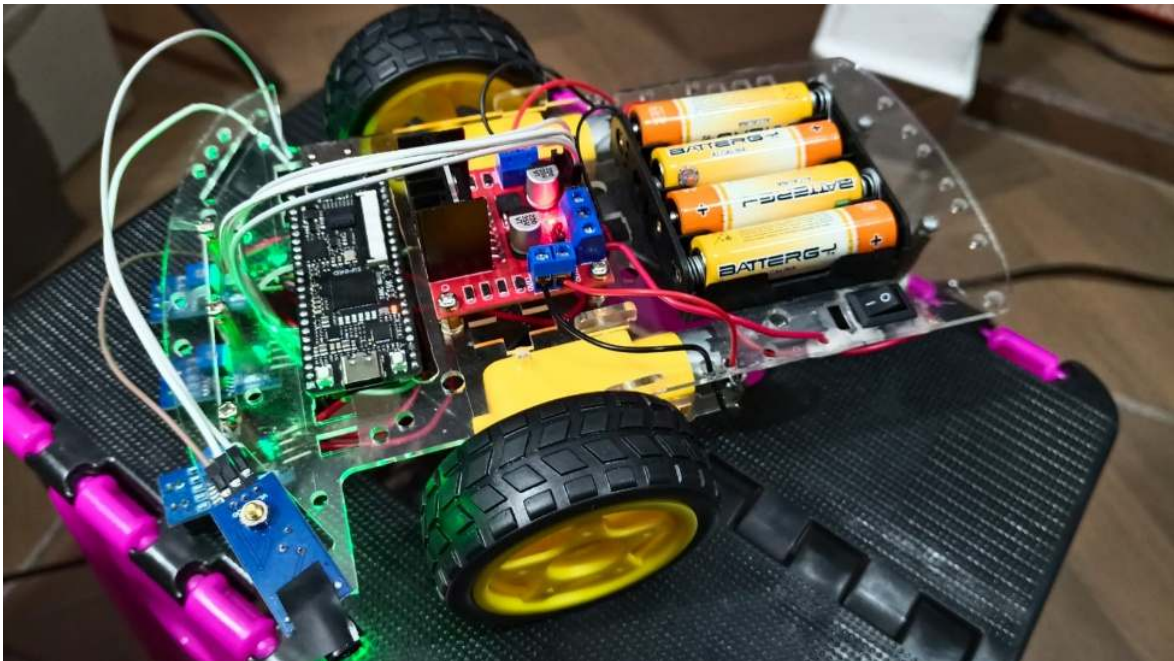
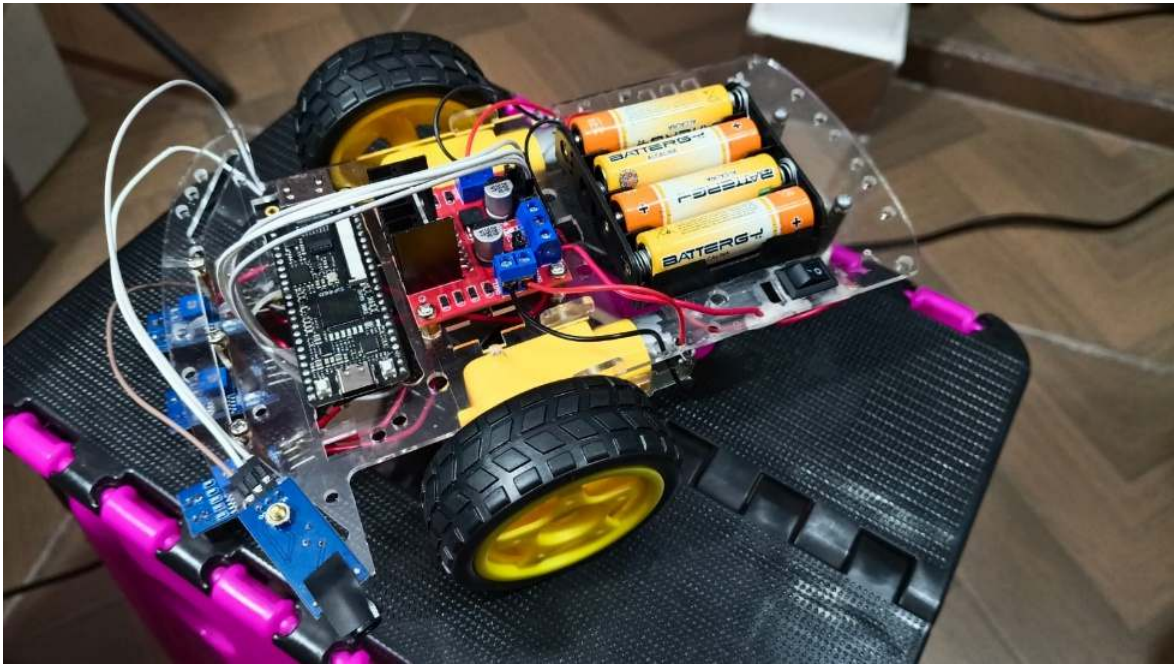
Para la asignación y acoplamiento de los pines físicos, se configuraron los puertos lógicos bajo las siguientes restricciones eléctricas en el hardware:

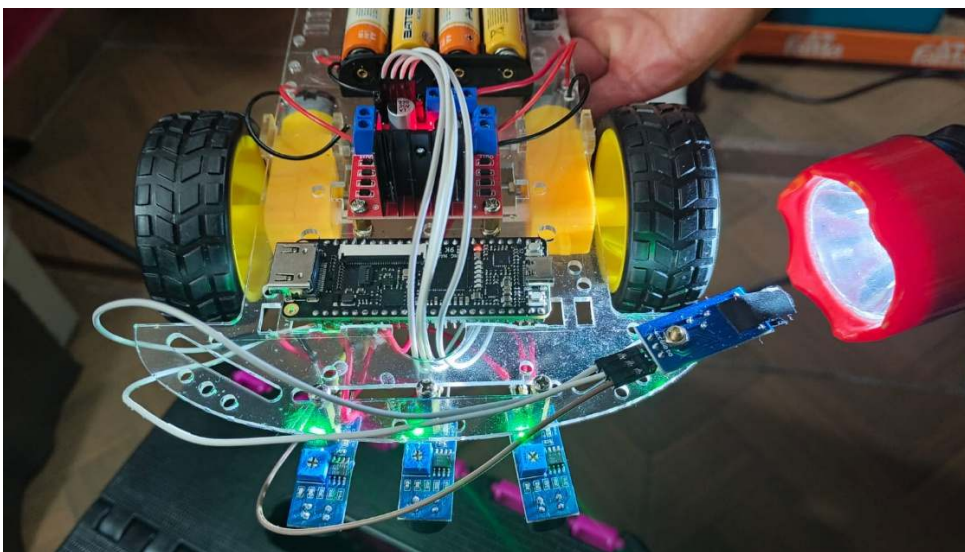
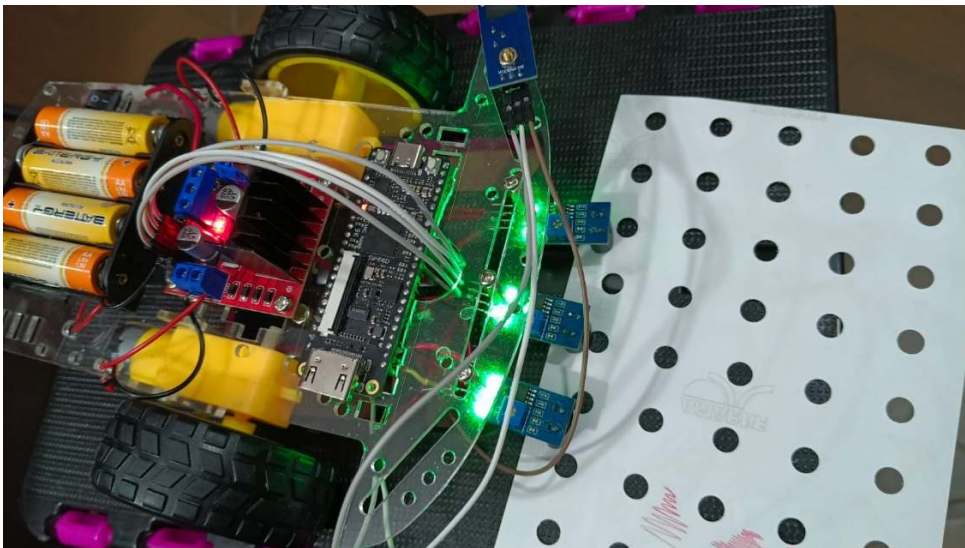
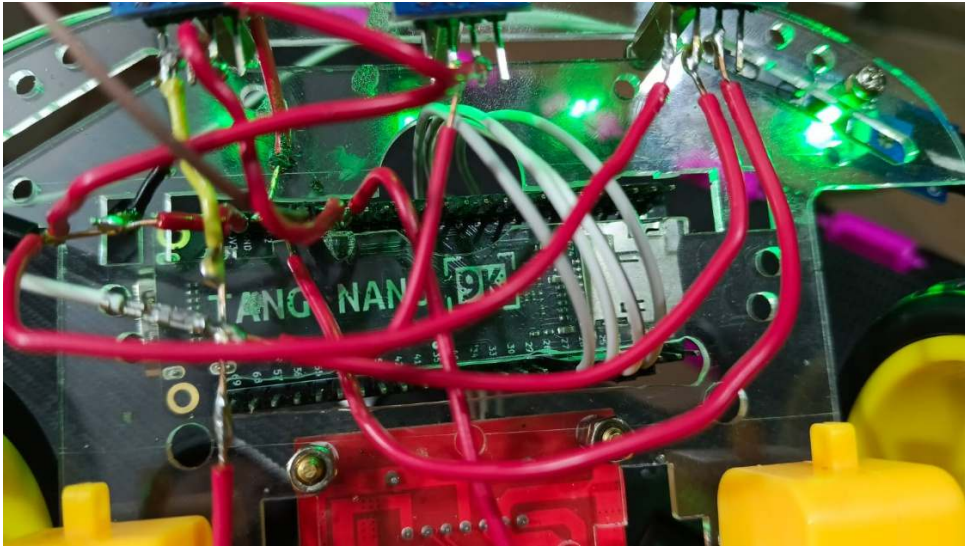
- **Estándar de Voltaje (IO Type = LVC MOS33):** Se seleccionó la lógica CMOS de bajo voltaje a **3.3 V** para todas las señales. Esto garantiza una compatibilidad eléctrica nativa y bidireccional entre los bancos de la FPGA Tang Nano 9K y los

módulos periféricos del carro (sensores ópticos TCRT5000, circuito LDR y las entradas lógicas de control del puente H L298N).

- **Modo de Polarización (Pull Mode = NONE):** Se deshabilitaron por completo las resistencias internas de *Pull-Up* y *Pull-Down* de la FPGA. Esta decisión evita interferencias o divisiones de tensión indeseadas, puesto que tanto la tarjeta de sensores como el módulo receptor del flash cuentan con sus propias resistencias de polarización físicas integradas en sus circuitos de acondicionamiento de señal.
- **Capacidad de Carga (Drive = 8 mA):** Las salidas digitales destinadas a excitar los optoacopladores o compuertas lógicas de entrada del Puente H (IN1 a IN4) se configuraron con una capacidad de corriente de drenaje de **8 mA**. Esta configuración de corriente es lo suficientemente robusta para conmutar los canales del driver de potencia de manera veloz sin comprometer ni estresar térmicamente los pines de silicio de la FPGA.

FOTOS DEL FPGA FUNCIONANDO:





CONCLUSIONES:

El desarrollo del presente proyecto permitió diseñar, sintetizar e implementar exitosamente un vehículo robótico seguidor de líneas, utilizando la FPGA Tang Nano 9K como unidad central de procesamiento y control. Mediante la descripción de hardware en lenguaje Verilog, se logró la integración de un sistema de adquisición de señales compuesto por tres transductores infrarrojos y un sensor óptico LDR, generando de manera síncrona las directivas lógicas necesarias para el guiado y navegación autónoma del móvil.

Durante la etapa de diseño arquitectónico, se consolidaron y aplicaron conceptos avanzados de los sistemas digitales, tales como la estructuración de lógica combinacional optimizada (evitando la generación de latches involuntarios mediante cláusulas default), el diseño de divisores de frecuencia por hardware, la modulación por ancho de pulsos (PWM), la asignación de restricciones físicas y eléctricas de entrada/salida (I/O Constraints en estándar LVCMOS33 y Pull Mode: NONE), y la programación directa sobre silicio. Asimismo, el uso de herramientas de simulación basada en vectores de prueba (Testbench) y síntesis lógica permitió validar formalmente la máquina de estados y el comportamiento del sistema antes de su despliegue físico.

A nivel de implementación práctica, el proyecto presentó importantes desafíos técnicos en su manufactura y calibración eléctrica. Por un lado, las soldaduras de las conexiones del circuito costaron un esfuerzo y tiempo considerables debido a la precisión requerida para evitar falsos contactos en los pines de la FPGA y el cableado de distribución. Por otro lado, durante las primeras pruebas de tracción, se manifestaron serios problemas al tratar de poner ambos motores en marcha a la vez, ya que la alta demanda de corriente inicial bloqueaba o reiniciaba el sistema de control debido a caídas de voltaje en la fuente de alimentación compartida.

Las evaluaciones experimentales posteriores demostraron que el vehículo posee la sensibilidad necesaria para decodificar las diferentes transiciones de la trayectoria en la pista, respondiendo con alta fidelidad mediante maniobras de avance rectilíneo, corrección de deriva, giros críticos y rutinas de reversa por pérdida de camino basadas en una lógica activa en bajo. Adicionalmente, el subsistema basado en el sensor LDR validó la efectividad de un mecanismo de enclavamiento seguro activado por un pulso lumínico (flash),

garantizando que el carro permanezca en modo de espera y arranque de forma controlada y segura.

La integración física en el chasis confirmó el acoplamiento y la compatibilidad electromecánica definitiva entre los niveles lógicos de la FPGA, la etapa de potencia comandada por el puente H L298N, los sensores y los actuadores de corriente continua (CC). Se constató experimentalmente que la implementación de señales PWM limitadas al 60% de ciclo de trabajo no solo resolvió las limitantes de inercia y velocidad excesiva en las curvas, sino que fue la clave para mitigar los efectos críticos de las caídas de tensión por la conmutación simultánea de los motores, estabilizando por completo el consumo energético general.

Finalmente, este proyecto demostró de manera práctica el alto rendimiento, la flexibilidad y la naturaleza reconfigurable de las FPGAs aplicadas a la robótica móvil y los sistemas de control embebido. Como trabajo futuro, se plantea la optimización de la plataforma mediante el incremento en la resolución del arreglo de sensores infrarrojos y la migración de la lógica combinacional hacia un algoritmo de control en lazo cerrado tipo PID (Proporcional-Integral-Derivativo), con el fin de suavizar la respuesta angular de los motores y lograr una navegación significativamente más precisa y robusta ante pistas de alta complejidad.